Tic-Tac-Toe Project

Course: Python Final Project

Team Members

Muhammad Abdul Hadi Qamar Hamza (Completed individually due to group unavailability)

Introduction

This project implements a complete Tic-Tac-Toe game in Python.

It includes:

- A console-based version (text mode)
- A graphical version using Tkinter (UI)
- Winner and draw detection
- Input validation
- A reset option in the UI

The project follows the assignment requirements: it uses functions, includes testing, and provides a user interface for bonus points.

Project Description

Tic-Tac-Toe is a two-player game played on a 3×3 grid.
Players take turns marking the grid with **X** or **O**.
A player wins if they place three of their marks in a row, column, or diagonal.
If all 9 squares are filled and no one wins, the game ends in a **draw**.

Algorithm Explanation

Board Representation

The board is stored as a list of 9 elements:

- Index 0–2: first row
- Index 3–5: second row

- Index 6–8: third row

Winner Checking

We check all winning combinations:

- Rows: (0,1,2), (3,4,5), (6,7,8)
- Columns: (0,3,6), (1,4,7), (2,5,8)
- Diagonals: (0,4,8), (2,4,6)

Game Flow (Console Version)

1. Display the board
2. Ask the current player (X or O) to choose a position (1–9)
3. Validate the input
4. Place the mark on the board
5. Check if the player has won
6. Check if the game is a draw
7. Switch player and repeat until win or draw

In [5]:
```
# Tic-Tac-Toe Console Version

def display_board(board):
    print("\n")
    print(board[0], "|", board[1], "|", board[2])
    print("--+---+--")
    print(board[3], "|", board[4], "|", board[5])
    print("--+---+--")
    print(board[6], "|", board[7], "|", board[8])
    print("\n")

def check_winner(board, player):
    wins = [
        (0,1,2), (3,4,5), (6,7,8),
        (0,3,6), (1,4,7), (2,5,8),
        (0,4,8), (2,4,6)
    ]
    for a, b, c in wins:
        if board[a] == board[b] == board[c] == player:
            return True
    return False

def check_draw(board):
    return " " not in board

def play_game():
    board = [" "] * 9
    current_player = "X"
```

```python
    while True:
        display_board(board)
        move = input(f"Player {current_player}, choose a position (1-9)

        if not move.isdigit() or not (1 <= int(move) <= 9):
            print("Invalid input. Try again.")
            continue

        move = int(move) - 1

        if board[move] != " ":
            print("Position already taken. Try again.")
            continue

        board[move] = current_player

        if check_winner(board, current_player):
            display_board(board)
            print(f"Player {current_player} wins!")
            break

        if check_draw(board):
            display_board(board)
            print("It's a draw!")
            break

        current_player = "O" if current_player == "X" else "X"

# IMPORTANT: Start the game
play_game()
```

```
  |   |
--+---+--
  |   |
--+---+--
  |   |



  | X |
--+---+--
  |   |
--+---+--
  |   |



  | X |
--+---+--
  | O |
--+---+--
  |   |
```

```
X | X |
--+---+--
  | O |
--+---+--
  |   |



X | X | O
--+---+--
  | O |
--+---+--
  |   |



X | X | O
--+---+--
X | O |
--+---+--
  |   |



X | X | O
--+---+--
X | O |
--+---+--
O |   |


Player O wins!
```

Tic-Tac-Toe UI Version (Tkinter)

To earn the bonus points, this project also includes a graphical user interface (UI) using Tkinter.

The UI:

- Shows a 3×3 grid of buttons
- Lets players click to place X or O
- Detects wins and draws
- Provides a Reset button to start a new game

In [10]:
```python
import tkinter as tk
from tkinter import messagebox

def reset_board():
    global board, buttons, current_player
```

```
    board = [" "] * 9
    current_player = "X"
    for btn in buttons:
        btn.config(text=" ", state="normal")

def check_winner_ui():
    wins = [
        (0,1,2), (3,4,5), (6,7,8),
        (0,3,6), (1,4,7), (2,5,8),
        (0,4,8), (2,4,6)
    ]
    for a, b, c in wins:
        if board[a] == board[b] == board[c] != " ":
            return True
    return False

def check_draw_ui():
    return " " not in board

def on_click(index):
    global current_player

    if board[index] == " ":
        board[index] = current_player
        buttons[index].config(text=current_player)

        if check_winner_ui():
            messagebox.showinfo("Winner", f"Player {current_player} win
            for btn in buttons:
                btn.config(state="disabled")
            return

        if check_draw_ui():
            messagebox.showinfo("Draw", "It's a draw!")
            return

        current_player = "O" if current_player == "X" else "X"

# UI Setup
board = [" "] * 9
current_player = "X"
buttons = []

window = tk.Tk()
window.title("Tic-Tac-Toe")

for i in range(9):
    btn = tk.Button(window, text=" ", font=("Arial", 24), width=5, heig
                    command=lambda i=i: on_click(i))
    btn.grid(row=i//3, column=i%3)
    buttons.append(btn)

reset_btn = tk.Button(window, text="Reset", font=("Arial", 16), command
reset_btn.grid(row=3, column=1)
```

```
window.mainloop()
```

Testing

The following tests were performed to make sure both the console version and the Tkinter UI version work correctly.

#	Scenario	Input / Action	Expected Result	Actual Result	Status
1	X wins (row)	X fills top row	Player X wins	Player X wins	Pass
2	O wins (column)	O fills first column	Player O wins	Player O wins	Pass
3	Draw	All 9 cells filled, no winner	Draw message	Draw message	Pass
4	Invalid move	Player selects an already used position	Error message shown	Error message shown	Pass
5	UI click	Clicking a button in Tkinter	X or O appears	Works correctly	Pass
6	UI reset	Press Reset button	Board clears	Board cleared	Pass

Conclusion

This project allowed me to build a complete Tic-Tac-Toe game using Python. We implemented both a console version and a graphical version using Tkinter, which made the game more interactive and user friendly. The program correctly handles player turns, detects wins and draws, and prevents invalid moves. The reset button in the UI also makes it easy to play multiple rounds.

Through this project, We practiced using functions, conditional logic, loops, and basic GUI programming. We also learned how to structure a Python project clearly and test different scenarios to make sure everything works as expected.

Overall, the project meets all the requirements of the assignment, including the bonus UI feature.

In [ ]: